

We follow the train-test split from the original MNIST dataset. The joint model is trained on 100,000 training examples ($m = 10$) and gets 99.4% accuracy on 1000 testing examples ($m = 50$). Note that the LeNet modules are optimized jointly with the reasoning about `AdjacentToNumber0`.

D REALIZATION OF HORN CLAUSE

In this section, we show that *NLM can realize a partial set of Horn clauses (Horn, 1951) in first-order logic (FOL), up to the limit of the NLM’s depth and breadth*. In NLMs, we consider only finite cases. Thus, there should not exist cyclic references of predicates among rules. The extension to support cyclic references is left as a future work. Throughout the proof, we always assume the depth, breadth and number of predicates of NLM are flexible and large enough to realize the demanding rules.

Here, we only prove the realization of a *definite clause*, i.e., a Horn clause with exactly one positive literal and a non-zero number of negative literals in FOL⁷. It can be written in the implication form is $\hat{p} \leftarrow p_1 \wedge p_2 \wedge \dots \wedge p_k$ (variables as arguments are implicitly universally quantified), where $\hat{p}$ is called the *head predicate* and $p_1, \dots, p_k$ are called *body predicates*. We group the variables appearing in the rule into three subsets: (1) variables that only appear in the head predicate, (2) variables that appear in the body predicates, and (3) variables that appear in both head and body predicates.

Consider as an example a chain-like rule: $\forall x_1 \forall x_2 \forall x_3 \forall x_4 \hat{p}(x_1, x_3, x_4) \leftarrow p_1(x_1, x_2) \wedge p_2(x_2, x_3)$. We rewrite the rule by classifying the variables:

$$\forall x_4 \left(\forall x_1 \forall x_3 \left(\hat{p}(x_1, x_3, x_4) \leftarrow \exists x_2 p_1(x_1, x_2) \wedge p_2(x_2, x_3) \right) \right) .$$

That is, we move all variables that only appear in *body predicates* to the right-hand side, and extract out all variables that only appear in the *head predicate*. We show how we can compositionally combines the computation units in NLMs to realize this rule, in the following 4 steps:

1. We first align the arity of the *body predicates* to include all variables that appear in at least one of the *body predicates* (including variables of set 2 and set 3). This could be done by a sequence of `Expand` operations (Eq. 3). In this example, we will create helper predicates to make the right-hand side of the rule as

$$\exists x_2 p'_1(x_1, x_2, x_3) \wedge p'_2(x_2, x_3, x_1),$$

where $p'_1(x_1, x_2, x_3) \triangleq p_1(x_1, x_2)$ and $p'_2(x_2, x_3, x_1) \triangleq p_2(x_2, x_3)$.

2. We use neural boolean logic (Eq. 1) to realize the boolean formula inside all quantification symbols. Moreover, we use the `Permute` operation to transpose the tensor representation so that all variables being quantified on the right-hand side appear as the last several variables in the derived predicate p' . Overall, we will derive another helper predicates,

$$p'(x_1, x_3, x_2) \triangleq p'_1(x_1, x_2, x_3) \wedge p'_2(x_2, x_3, x_1),$$

3. We use the `Reduce` operation to add quantifiers to the right-hand side (i.e., to the p' predicate). We will get:

$$p''(x_1, x_3) \triangleq \exists x_2 p'(x_1, x_3, x_2) = \exists x_2 p'_1(x_1, x_2, x_3) \wedge p'_2(x_2, x_3, x_1),$$

4. Finally, we use the `Expand` operation (Eq. 3] to add variables that only appear in the *head predicate* to the derived predicate:

$$\hat{p}(x_1, x_3, x_4) \triangleq p''(x_1, x_3).$$

Note that, all variables appeared in the *head predicate* are implicitly universally quantified. This is consistent with our setting, since all rules in NLMs are lifted.

Overall, a symbolic rule written as a Horn clause can be realized by NLMs as a computation flow which starts from multiple expansions followed by a neural boolean rule and multiple reductions, and ends with a set of expansions.

⁷As for other types of Horn clauses: *facts* are realized as the tensor representations of predicates, while we implicitly views the output of NLMs as the *goal clauses*.

Next, we show that the forward propagation of NLMs realizes the forward chaining of a set of Horn clauses. Following the notation in Evans & Grefenstette (2018), the forward chaining starts from a set of initial *facts*, which are essentially the grounding of base predicates. The forward chaining process sequentially applies rules over the *fact* set, and concludes new *facts*. In NLM, we represent facts as the $\mathcal{U}$ -grounding of predicates.

If we consider a set of rules that does not have recursive references, all rules can be sorted in an topological order $\mathcal{R} = (r_1, r_2, \dots, r_k)$. We only allow references of r_i from r_j , where $i < j$. Without loss of generality, we assume that the grounding of r_k is of interest. Given the topologically resolved set of rules $\mathcal{R}$, we build a set of NLMs where each NLM realizes a specific rule r_i . By stacking the NLMs sequentially, we can conclude r_k . As a side note, for multiple rules referring to the same *head predicate* $\hat{p}$, they implicitly indicate the logical disjunction ($\vee$) of the rules. We can rename these head predicates as $\hat{p}_1, \hat{p}_2, \dots$, and use an extra NLM to implement the logical disjunction of all $\hat{p}_i$'s.

E SAMPLE BLOCKS WORLD RULES

This example shows a complex reasoning in the seemingly simple Blocks World domain, which can be solved by our NLMs but requires great efforts of create manual rules by *human experts* in contrast.

Suppose we are interested in knowing *whether a block should be moved* in order to reach the target configuration. Here, a block should be moved if (1) it is moveable; and (2) there is at least one block below it that does not match the target configuration. Call the desired predicate "ShouldMove(x)".

Input Relations. (Specified in the last paragraph of Section 3.4) SameWorldID, SmallerWorldID, LargerWorldID; SameID, SmallerID, LargerID; Left, SameX, Right, Below, SameY, Above. The relations are given on all pairs of objects across both worlds.

Here is one way to produce the desired predicate by defining several helper predicates, designed by "human experts":

1. $\text{IsGround}(x) \leftarrow \forall y \text{ Above}(y, x)$
2. $\text{SameXAbove}(x, y) \leftarrow \text{SameWorldID}(x, y) \wedge \text{SameX}(x, y) \wedge \text{Above}(x, y)$
3. $\text{Clear}(x) \leftarrow \forall y \neg \text{SameXAbove}(y, x)$
4. $\text{Moveable}(x) \leftarrow \text{Clear}(x) \wedge \neg \text{IsGround}(x)$
5. $\text{InitialWorld}(x) \leftarrow \forall y \neg \text{SmallerWorldID}(y, x)$
6. $\text{Match}(x, y) \leftarrow \neg \text{SameWorldID}(x, y) \wedge \text{SameID}(x, y) \wedge \text{SameX}(x, y) \wedge \text{SameY}(x, y)$
7. $\text{Matched}(x) \leftarrow \exists y \text{ Match}(x, y)$
8. $\text{HaveUnmatchedBelow}(x) \leftarrow \exists y \text{ SameXAbove}(x, y) \wedge \neg \text{Matched}(y)$
9. $\text{ShouldMove}(x) \leftarrow \text{InitialWorld}(x) \wedge \text{Moveable}(x) \wedge \text{HaveUnmatchedBelow}(x)$

We can also write the logic forms in one line: $\text{ShouldMove}(x) \leftarrow (\forall y \neg \text{SmallerWorldID}(y, x)) \wedge (\forall y \neg (\text{SameWorldID}(y, x) \wedge \text{SameX}(y, x) \wedge \text{Above}(y, x))) \wedge \neg (\forall y \text{ Above}(y, x)) \wedge ((\exists y \text{ SameWorldID}(x, y) \wedge \text{SameX}(x, y) \wedge \text{Above}(x, y)) \wedge \neg (\exists z \neg \text{SameWorldID}(y, z) \wedge \text{SameID}(y, z) \wedge \text{SameX}(y, z) \wedge \text{SameY}(y, z)))$.

Note that this is only a part of logic rules needed to complete the Blocks World challenge. The learner also needs to figure out where should the block be moved onto. The proposed NLM can learn policies that solve the Blocks World from the sparse reward signal indicating only whether the agent has finished the game. More importantly, the learned policy generalizes well to larger instances (consisting more blocks).

F IMPLEMENT NLM IN TENSORFLOW

The following python code contains a minimal implementation for one Neural Logic Machines layer with breadth equals 3 in TensorFlow. The `neural_logic_layer_breadth3` is the main function. The syntax is highlighted and is best viewed in color.

```

1  from itertools import permutations
2  import tensorflow as tf
3  from tensorflow.layers import dense
4
5  def expand(input, M):
6      """Expands input at its second last dimension (e.g., [B, ...,
7      ↪ Ni, Nj] to [B, ..., Ni, M, Nj]) by replicating tensors."""
8      ndims = input.get_shape().ndims + 1
9      multiples = [M if i == ndims - 2 else 1 for i in range(ndims)]
10     return tf.tile(tf.expand_dims(input, -2), multiples)
11
12 def reduce(input, M):
13     """Reduces max and min at the second last dimension, except for
14     ↪ diagonal elements."""
15     mask = _reduce_mask(input, M) [tf.newaxis, ..., tf.newaxis]
16     return tf.concat([
17         tf.reduce_max(input * mask, -2),
18         tf.reduce_min(input * mask + (1 - mask), -2)
19     ], -1)
20
21 def neural_logic(input, hidden_dim):
22     """An MLP layer applied on permutations of the input."""
23     return dense(_input_permutations(input), hidden_dim,
24                 ↪ activation=tf.sigmoid)
25
26 def neural_logic_layer_breath3(input0, input1, input2, input3, M,
27     ↪ hidden_dim, residual):
28     """A neural logic layer with breath 3.
29     Args:
30         input0: float Tensor of shape [B, hidden_dim], nullary
31         ↪ predicates.
32         input1: float Tensor of shape [B, M, hidden_dim], unary
33         ↪ predicates.
34         input2: float Tensor of shape [B, M, M, hidden_dim], binary
35         ↪ predicates.
36         input3: float Tensor of shape [B, M, M, M, hidden_dim],
37         ↪ ternary predicates.
38         M: int, number of objects.
39         hidden_dim: int, hidden dimension.
40         residual: boolean, use the residual link or not.
41     Returns:
42         4 float Tensors, output nullary, unary, binary ternary
43         ↪ predicates respectively.
44     """
45     agg0 = tf.concat([input0, reduce(input1, M)], -1)
46     agg1 = tf.concat([input1, expand(input0, M), reduce(input2,
47         ↪ M)], -1)
48     agg2 = tf.concat([input2, expand(input1, M), reduce(input3,
49         ↪ M)], -1)
50     agg3 = tf.concat([input3, expand(input2, M)], -1)
51     outputs = [neural_logic(x, hidden_dim) for x in [agg0, agg1,
52         ↪ agg2, agg3]]
53     if residual:
54         outputs = [tf.concat([x, y], -1) for x, y in zip(outputs,
55         ↪ [input0, input1, input2, input3])]
56     return outputs
57
58 def _reduce_mask(input, M):

```